



UNIVERSIDADE
LUSÓFONA

Aplicação web para gestão da qualidade de implementação de User Stories

Trabalho Final de curso

Relatório Intercalar 2º Semestre

Letizia Nunes, a22407143, LEI

Laércio Santos, a22401473, LEI

Orientador: Luís Gomes

Coorientador: José Brás

Entidade Externa: CGI

Departamento de Engenharia Informática da Universidade Lusófona

Centro Universitário de Lisboa

16/10/2025

www.ulusofona.pt

Aviso de confidencialidade (NDA)

Este trabalho está abrangido por Acordo de Não Divulgação (NDA); qualquer disponibilização pública fica condicionada à eliminação/anonimização de informação confidencial e/ou à autorização escrita prévia da CGI; a versão pública depositada será necessariamente expurgada.

Direitos de cópia

Aplicação web para gestão da qualidade de implementação de User Stories, Copyright de Letizia Nunes e Laércio Santos, ULHT.

A Escola de Comunicação, Arquitetura, Artes e Tecnologias da Informação (ECATI) e a Universidade Lusófona de Humanidades e Tecnologias (ULHT) têm o direito, perpétuo e sem limites geográficos, de arquivar e publicar esta dissertação através de exemplares impressos reproduzidos em papel ou de forma digital, ou por qualquer outro meio conhecido ou que venha a ser inventado, e de a divulgar através de repositórios científicos e de admitir a sua cópia e distribuição com objetivos educacionais ou de investigação, não comerciais, desde que seja dado crédito ao autor e editor.

Resumo

A implementação eficiente de user stories é crucial para o sucesso de projetos de desenvolvimento de software em ambientes ágeis. Diante da limitação de ferramentas genéricas, este trabalho propõe o desenvolvimento de uma aplicação web especializada para a gestão e acompanhamento da implementação de User Stories.

A solução segue uma arquitetura cliente-servidor, com o desenvolvimento dividido entre equipes especializadas de frontend e backend. Este relatório foca-se no contributo da equipa de frontend, responsável pela conceção e desenvolvimento da interface de utilizador (UI) e experiência do utilizador (UX) da aplicação web. O objetivo passa por criar uma interface intuitiva e reativa que facilite o acompanhamento em tempo real do estado de implementação de cada User Story.

Este projeto é desenvolvido em parceria com a entidade externa CGI, com objetivo resolver uma necessidade concreta.

Palavras-chave: Gestão de User Stories, Frontend, Aplicação Web, Interface de Utilizador, Experiência de Utilizador, Metodologias Ágeis.

Abstract

The efficient implementation of User Stories is crucial for the success of software development projects in agile environments. Faced with the limitations of generic tools, this work proposes the development of a specialized web application for the management and monitoring of user story implementation.

The solution follows a client-server architecture, with development split between specialized frontend and backend teams. This report focuses on the contribution of the frontend team responsible for the design and development of the application's User Interface (UI) and User Experience (UX). The objective is to create an intuitive and reactive interface that facilitates the real-time tracking of each User Story's implementation status.

This project is developed in partnership with the external entity CGI, aiming to address a concrete need identified in their operational context.

Key-words: User Story Management, Frontend, Web Application, User Interface, User Experience, Agile Methodologies.

Índice

Resumo.....	iii
Abstract	iv
Índice.....	1
Lista de Figuras	3
Lista de Tabelas	4
Lista de Siglas.....	5
1 Introdução.....	6
1.1 Enquadramento	6
1.2 Motivação e Identificação do Problema	6
1.3 Objetivos	7
1.3.1 Objetivo geral	7
1.3.2 Objetivo do frontend.....	7
1.3.3 Objetivo do backend	7
1.4 Estrutura do Documento.....	7
2 Pertinência e Viabilidade.....	8
2.1 Pertinência	8
2.2 Viabilidade	9
2.3 Análise Comparativa com Soluções Existentes	10
2.3.1 Soluções existentes	10
2.3.2 Análise de benchmarking	10
2.4 Proposta de inovação e mais-valias.....	10
3 Especificação e Modelação	11
3.1 Análise de Requisitos.....	11
3.1.1 Enumeração de Requisitos.....	11
3.1.2 Descrição detalhada dos requisitos principais	13
3.2 Modelação	15
3.3 Protótipos de Interface	15
4 Solução Proposta.....	16
4.1 Apresentação	16
4.2 Arquitetura.....	16
4.3 Tecnologias e Ferramentas Utilizadas	16
4.3.1 Estratégia de Desenvolvimento do Frontend.....	16

4.4	<i>Fluxos de interação e integração</i>	18
4.4.1	Fluxo de autenticação e armazenamento de informações	18
4.4.2	Fluxo de interação (User Stories)	18
4.4.3	Fluxo de integração (user stories)	18
4.4.4	Fluxo de interação (submissão para revisão com IA)	18
4.4.5	Fluxo de integração (submissão para revisão com IA, hipótese)	19
4.4.6	Responsabilidades (frontend e backend)	19
4.5	<i>Features e Enhancements relevantes</i>	19
4.5.1	Sistema de Gamificação	19
4.5.2	Página Friends (dependente de tempo e progresso)	19
4.6	<i>Abrangência</i>	20
4.7	<i>Componentes</i>	20
4.8	<i>Interfaces</i>	21
5	Testes e Validação	25
5.1	<i>Abordagem de validação</i>	25
5.2	<i>Ferramentas utilizadas</i>	25
5.3	<i>Cobertura de Testes</i>	26
5.3.1	Renderização	26
5.3.2	Validação de Campos	26
5.3.3	Estado de Carregamento	26
5.3.4	Caminho de Sucesso	26
5.3.4	Caminho de Erro	26
5.4	<i>Isolamento e Mocking</i>	27
5.5	<i>Testes de Integração com a API</i>	27
6	Método e Planeamento	29
	Bibliografia	30
	Glossário	31
	Anexo A - Formulário de declaração de uso de ferramentas de Inteligência Artificial a anexar ao relatório	32

Lista de Figuras

Figura 1 - Mapa aplicacional	15
Figura 2 - Home page sem login	21
Figura 3 - Home page com login	22
Figura 4 - Input page.....	22
Figura 5 - User stories page	23
Figura 6 - User story page.....	23
Figura 7 - User profile showing yearly activity and unlocked achievements	24
Figura 8 – Histórico de testes realizados no Postman.....	27
Figura 9 - POST bem-sucedido para avaliação de uma UST	28
Figura 10 - POST bem-sucedido para avaliação de uma UST (continuação)	28
Figura 11 – Diagrama de Gantt.....	29
Figura 12 - Diagrama de Gantt (continuação)	29

Lista de Tabelas

Tabela 1 - Benchmarking entre soluções e a nossa proposta	10
Tabela 2 - Catálogo Inicial de Requisitos do Frontend	11
Tabela 3 - Tecnologias e Ferramentas Utilizadas no Frontend.....	17
Tabela 4 - Ferramentas de Teste Utilizadas.....	25

Lista de Siglas

- AC** — *Acceptance Criteria* (**Critérios de Aceitação**)
- AI** — *Artificial Intelligence* (**Inteligência Artificial**)
- API** — *Application Programming Interface* (**Interface de Programação de Aplicações**)
- BD** — *Database* (**Base de Dados**)
- CSS** — *Cascading Style Sheets* (**Folhas de Estilo em Cascata**)
- Git** — *Version Control System Git* (**Sistema de Controlo de Versão Git**)
- HTML** — *HyperText Markup Language* (**Linguagem de Marcação de Hipertexto**)
- HTTP** — *HyperText Transfer Protocol* (**Protocolo de Transferência de Hipertexto**)
- IDE** — *Integrated Development Environment* (**Ambiente de Desenvolvimento Integrado**)
- JSON** — *JavaScript Object Notation* (**Notação de Objetos JavaScript**)
- MSW** — *Mock Service Worker* (**Simulador de Serviços Web**)
- OAuth** — *Open Authorization* (**Autorização Aberta**)
- PDF** — *Portable Document Format* (**Formato Portátil de Documento**)
- PO** — *Product Owner* (**Dono do Produto**)
- REST** — *Representational State Transfer* (**Transferência de Estado Representacional**)
- RF** — *Functional Requirement* (**Requisito Funcional**)
- RNF** — *Non-Functional Requirement* (**Requisito Não Funcional**)
- SP** — *Story Points* (**Pontos da História**)
- SQL** — *Structured Query Language* (**Linguagem de Consulta Estruturada**)
- TFC** — *Final Course Project* (**Trabalho de Fim de Curso**)
- UI** — *User Interface* (**Interface do Utilizador**)
- UX** — *User Experience* (**Experiência do Utilizador**)
- US/UST** — *User Story* (**História de Utilizador**)
- VCS** — *Version Control System* (**Sistema de Controlo de Versões**)
- VSCo** — *Visual Studio Code* (**Visual Studio Code — Editor de Código**)

1 Introdução

1.1 Enquadramento

No panorama atual do desenvolvimento de software, as metodologias ágeis afirmaram-se como um padrão da indústria, com as User Stories a funcionarem como a unidade fundamental para a definição de requisitos e funcionalidades [1]. A eficácia de uma equipa ágil está, portanto, intrinsecamente ligada à sua capacidade de gerir de forma clara, transparente e eficiente o ciclo de vida destas mesmas user stories – desde a sua conceção até à implementação e teste. No entanto, muitas organizações depararam-se com a limitação de ferramentas de gestão de projeto genéricas (ex.: Jira, Trello), que, embora poderosas, nem sempre se adaptam de forma ideal aos fluxos de trabalho e terminologias específicas de cada equipa ou projeto.

Este trabalho enquadra-se no desenvolvimento de uma solução para a proposta feita pela entidade externa CGI [2], com conhecimentos adquiridos na Universidade Lusófona [3].

1.2 Motivação e Identificação do Problema

Em ambientes de desenvolvimento ágil de alta intensidade, como os operados pela entidade parceira CGI, a gestão eficiente do ciclo de vida das user stories e a garantia da qualidade do código são fundamentais para o sucesso dos projetos. Atualmente, estes dois pilares – gestão e qualidade – operam frequentemente em silos separados. O processo padrão exige que o desenvolvedor interprete a user story (gestão), implemente a funcionalidade e, apenas num momento posterior e distinto, submeta o código para revisão manual de pares (qualidade).

Esta desconexão cria um bottleneck crítico no fluxo de desenvolvimento. A verificação da qualidade e do alinhamento com os requisitos acontece tardiamente, dependente da disponibilidade e perceção subjetiva de outros elementos da equipa. O tempo de espera por este feedback prejudica a produtividade e aumenta significativamente o risco de retrabalho, o que compromete a eficácia da gestão do projeto.

Surge, assim, a necessidade clara de uma solução que una estes dois mundos. O problema central identificado é a falta de integração automatizada entre a gestão do ciclo de vida das user stories e a verificação objetiva da qualidade e conformidade do código correspondente. Este projeto propõe-se a resolver esta lacuna através do desenvolvimento de uma aplicação que não só facilita a gestão das tarefas, como introduz um controlo de qualidade automatizado e imediato no ponto de implementação.

1.3 Objetivos

1.3.1 Objetivo geral

Desenvolver uma aplicação web inovadora que recebe uma user story como input, processa-a através de um motor de Inteligência Artificial (IA) e, para além de uma bateria de testes que valida o cumprimento dos requisitos, fornece ao desenvolvedor um feedback construtivo e detalhado. Este feedback inclui a identificação do que está correto, sugestões de melhoria e recomendações de boas práticas para a produção de código limpo e sustentável.

1.3.2 Objetivo do frontend

1. Conceber e desenvolver uma interface de utilizador (UI) intuitiva, acessível e responsiva, que facilite a submissão de user stories e a visualização clara do feedback gerado pela IA.
2. Garantir uma experiência de utilizador (UX) fluida e motivadora, que apresentará a informação providenciada pela API de forma clara, organizada e construtiva.
3. Implementar a aplicação web frontend que consuma a API RESTful, que possibilitará uma interação dinâmica e eficiente com o utilizador.
4. Explorar e integrar elementos de gamificação (como sistemas de badges, níveis ou barras de progresso) para manter o desenvolvedor envolvido e focado na melhoria contínua da qualidade do seu código.

1.3.3 Objetivo do backend

O objetivo do backend será detalhado no **capítulo 4.4.6**

1.4 Estrutura do Documento

Este relatório está organizado da seguinte forma:

- **Secção 1 - Introdução:** Apresenta o enquadramento do problema, a motivação para a realização do projeto, os objetivos gerais e específicos e a estrutura do documento.
- **Secção 2 - Pertinência e Viabilidade:** Discute a relevância do projeto e a sua viabilidade técnica, económica e social.
- **Secção 3 - Especificação e Modelação:** Descreve os requisitos, modelação e protótipos da aplicação web.
- **Secção 4 - Solução Proposta:** Detalha a arquitetura, as tecnologias e ferramentas utilizadas, bem como os componentes da solução.
- **Secção 6 - Método e Planeamento:** Expõe o cronograma e o método de trabalho adotado.

2 Pertinência e Viabilidade

2.1 Pertinência

A presente proposta consiste no desenvolvimento do frontend de uma aplicação web inovadora, destinada não só à gestão, mas também à avaliação automatizada da qualidade da implementação de user stories. A sua pertinência é particularmente relevante no contexto atual de desenvolvimento de software, onde as metodologias ágeis exigem ciclos de entrega curtos, critérios de aceitação rigorosos e um alinhamento constante entre product owners e developers.

A capacidade de verificar automaticamente, através de Inteligência Artificial (IA), se o código desenvolvido cumpre integralmente os requisitos de cada user story representa um avanço significativo. Esta inovação tem o potencial de aumentar drasticamente a eficiência dos processos, a fiabilidade das entregas e a qualidade final do software produzido.

A pertinência do projeto é ainda validada pela entidade parceira, a CGI, que identificou uma lacuna operacional concreta nos seus fluxos de trabalho. Apesar de utilizarem ferramentas consolidadas como o GitHub e o Azure DevOps para gestão de código e tarefas, não existe atualmente nenhuma solução que avalie, de forma centralizada e automatizada, a conformidade do código com os critérios funcionais definidos. Esta aplicação surge, portanto, para colmatar esta falha, com a introdução de maior rastreabilidade, transparência e objetividade numa fase crítica do processo de desenvolvimento.

2.2 Viabilidade

A viabilidade do projeto é analisada sob várias dimensões para confirmar a sua exequibilidade no âmbito do Trabalho Final de Curso.

Viabilidade Técnica

Foi concedida liberdade total na seleção de tecnologias, o que permitiu a adoção de soluções modernas e adequadas ao estado da arte. Para o frontend, está previsto o uso de Next.js, um framework robusto e amplamente suportado. A integração com serviços externos, como a API do GitHub para autenticação (OAuth) e gestão de repositórios, é tecnicamente suportada através de APIs bem documentadas e estáveis. A equipa de backend, responsável pelo processamento avançado e integração com o modelo de IA, está a desenvolver uma API RESTful, o que assegura uma separação clara de responsabilidades. A primeira fase de desenvolvimento inclui a criação de um protótipo de baixa fidelidade, o que permite uma validação rápida dos fluxos de utilizador e uma iteração eficiente antes da implementação, para assim mitigar riscos técnicos.

Viabilidade Económica

O desenvolvimento da aplicação não implica custos significativos, uma vez que as tecnologias selecionadas são open-source e os serviços externos previstos (como a API do GitHub) oferecem tiers gratuitos adequados para o escalamento inicial.

Viabilidade Social e Operacional

O público-alvo inicial são developers, um grupo com elevada literacia digital e familiaridade com os conceitos e ferramentas inerentes ao projeto. Esta característica garante uma curva de adoção rápida e um feedback qualificado durante a fase de testes. A ausência de um requisito de divulgação pública inicial simplifica o lançamento, o que reduz encargos legais e administrativos.

2.3 Análise Comparativa com Soluções Existentes

2.3.1 Soluções existentes

No mercado, coexistem várias ferramentas que suportam a verificação de critérios de aceitação (AC) e a automatização de testes, tais como Robot Framework, Cucumber, FitNesse, Concoction e abordagens baseadas em Processamento de Linguagem Natural (NLP) e IA (ex.: CJBA). Embora estas soluções cubram domínios como testes automatizados, Behavior-Driven Development (BDD) e geração de casos de teste, apresentam limitações críticas: exigem, na sua maioria, uma significativa intervenção técnica ou não oferecem uma interface verdadeiramente intuitiva e direcionada ao Product Owner.

2.3.2 Análise de benchmarking

Tabela 1 - Benchmarking entre soluções e a nossa proposta

Solução	Verificação automática de AC	Geração automática de testes	Integração CI/CD	Interface PO-friendly	Gamificação
Robot Framework	X		X		
Cucumber	X		X	X	
FitNesse	X		X	X	
Concoction	X		X		
CiRA (NLP/IA)	X	X	X		
App (proposta)	X	X	X	X	X

2.4 Proposta de inovação e mais-valias

A solução proposta distingue-se pela sua abordagem integrada, combina a verificação automática de critérios de aceitação com a geração automática de testes, integração CI/CD e uma interface intuitiva direcionada ao Product Owner. Inclui ainda funcionalidades de produto inovadoras, como sistemas de gamificação e filtros pessoais de desempenho — elementos ausentes ou pouco desenvolvidos nas soluções concorrentes analisadas

3 Especificação e Modelação

3.1 Análise de Requisitos

3.1.1 Enumeração de Requisitos

A tabela seguinte apresenta o catálogo inicial de requisitos identificados para o frontend da aplicação. Esta lista representa os requisitos de alto nível identificados nesta fase de planeamento, e será progressivamente refinada e detalhada ao longo do desenvolvimento do projeto, à medida que novas necessidades forem identificadas e os feedbacks dos protótipos forem incorporados.

Tabela 2 - Catálogo Inicial de Requisitos do Frontend

ID	Descrição	Tipo	Prioridade	Impacto	CrITÉrios de Aceitação
RF-01	Autenticação via OAuth GitHub	Funcional	Alta	5	Login com GitHub funciona; token validado; sessão iniciada.
RF-02	Criar User Stories no frontend	Funcional	Alta	5	Formulário permite criar US com título, descrição, AC e SP.
RF-03	Submeter código para revisão	Funcional	Alta	5	Envio de repo/commitHash para API; erros mostrados ao utilizador.
RF-04	Visualização de User Stories	Funcional	Alta	4	Lista US com estado, filtro e pesquisa.
RF-05	Visualização de resultados de revisão	Funcional	Alta	5	Após review, frontend exibe score, relatório e AC avaliados.
RF-06	Dashboard do utilizador	Funcional	Alta	4	Mostra XP, nível, últimas reviews, últimas US.
RF-07	Página de detalhes da US	Funcional	Alta	4	Exibe AC, SP, estado, revisões associadas.
RF-08	Mockups implementados (UI principal)	Funcional	Alta	4	Todas as páginas mockadas têm equivalente funcional no frontend.
RF-09	Histórico de avaliações	Funcional	Média	4	Listagem com datas, scores, commits, estado da análise.
RF-10	Exportar relatório de revisão (PDF/JSON)	Funcional	Média	3	Botão exportar disponível; ficheiro gerado com sucesso.

RF-11	Barra de progresso de XP	Funcional	Média	3	Mostra percentagem para o próximo nível; atualiza após review.
RF-12	Exibir badges desbloqueados	Funcional	Média	4	Badges visíveis no perfil; ocultar os não desbloqueados.
RF-13	Conversão de SP em XP (utilizador)	Funcional	Média	4	XP atualizado automaticamente após cada US completada.
RF-14	Exibir nível do utilizador	Funcional	Média	4	Mostrado no dashboard e perfil.
RF-15	Página Friends	Funcional	Média	3	Lista amigos com XP, nível, badges.
RF-16	Leaderboard global	Funcional	Média	3	Ordenação por XP/nível; top 10.
RF-17	Leaderboard entre amigos	Funcional	Baixa	2	Mostra ranking apenas dos amigos.
RNF-01	Responsividade total da interface	Não Funcional	Alta	5	Aplicação ajusta-se a desktop, tablet e mobile.
RNF-02	Tempo de carregamento inferior a 2s	Não Funcional	Alta	4	Páginas principais carregam <2s em 4G.
RNF-03	Feedback visual imediato	Não Funcional	Alta	4	Toasts e loaders sempre que há request à API.
RNF-04	Segurança do token OAuth	Não Funcional	Alta	5	Token nunca armazenado em local inseguro (ex.: localStorage).
RNF-05	Erros apresentados de forma humanizada	Não Funcional	Alta	4	Mensagens claras como "Commit inválido", "API indisponível"
RNF-06	Conformidade com a arquitetura REST	Não Funcional	Média	4	Todas as chamadas seguem métodos e payloads documentados.
RNF-09	Acessibilidade AA (WCAG)	Não Funcional	Baixa	2	Contraste, alt-text, labels e navegação por teclado.

Impacto:

- 5 Crítico/Máximo - Essencial para o sucesso
- 4 Alto - Muito importante
- 3 Médio - Importante
- 2 Baixo - Desejável
- 1 Mínimo - Pouco impacto

3.1.2 Descrição detalhada dos requisitos principais

- **RF-01 — Autenticação via GitHub OAuth**
- **Descrição:** O sistema deve permitir que o utilizador se autentique com utilização da sua conta GitHub, sem necessidade de criar credenciais específicas para a aplicação.
- **Objetivo:** Simplificar o processo de acesso e garantir segurança através de um provider confiável.
- **Critérios de Aceitação:**
 - Deve existir um botão “Login with GitHub” na página inicial.
 - O utilizador deve ser redirecionado para a página de autenticação do GitHub.
 - Após autenticação bem-sucedida, o utilizador deve ser enviado para o dashboard.
 - O token de sessão deve ser armazenado de forma segura.
- **Dependências:** API do GitHub; componente de autenticação no frontend.
- **RF-02 — Criar User Stories**
- **Descrição:** O sistema deve permitir ao utilizador criar novas User Stories através de um formulário estruturado com título, descrição, critérios de aceitação e Story Points.
- **Objetivo:** Fornecer ao utilizador a capacidade de submeter US que serão posteriormente avaliadas pelo backend.
- **Critérios de Aceitação:**
 - Deve existir um formulário para criação de US com todos os campos obrigatórios.
 - Os campos obrigatórios devem ser validados antes da submissão.
 - Após criação, a US deve aparecer imediatamente na lista de User Stories.
- **Dependências:** Componente de formulário; API do Backend para registar US.

- **RF-03 — Listar User Stories**

- **Descrição:** O sistema deve apresentar uma lista com todas as User Stories criadas pelo utilizador, inclui estado, título, Story Points e botão de ação.
- **Objetivo:** Permitir ao utilizador consultar rapidamente as US existentes e aceder às ações relacionadas (editar, ver resultados).
- **Critérios de Aceitação:**
 - A lista deve mostrar todas as US carregadas a partir da API.
 - Deve apresentar título, estado, SP e data de criação.
- **Dependências:** API do Backend para listagem de US.

- **RF-04 — Submeter Código para Revisão**

- **Descrição:** O sistema deve permitir ao utilizador submeter um repositório GitHub ou commit hash para avaliação automática pelo backend.
- **Objetivo:** Integrar o frontend com o motor de avaliação do backend, para permitir análise automática do código.
- **Critérios de Aceitação:**
 - Deve existir um campo para inserir URL do repositório ou commit.
 - A submissão deve enviar a referência do código e a US correspondente para a API.
 - O utilizador deve ser notificado quando a submissão for aceite.
 - Erros devem ser exibidos claramente (ex.: “Commit inválido”).
- **Dependências:** API REST do Backend; estrutura da US.

- **RF-05 — Visualizar Resultado da Revisão**

- **Descrição:** O sistema deve apresentar ao utilizador o resultado da avaliação efetuada pelo backend, inclui score, critérios cumpridos e recomendações.
- **Objetivo:** Fornecer visibilidade clara sobre a qualidade da implementação em relação aos critérios da US.
- **Critérios de Aceitação:**
 - Após a análise estar concluída no backend, o frontend deve apresentar o relatório.
 - O relatório deve incluir score geral e estado de cada critério.
 - Deve ser possível aceder ao relatório a partir da lista de US.
- **Dependências:** Endpoint do backend com resultados da avaliação.

- **RF-06 — Dashboard Inicial (MVP)**
- **Descrição:** O sistema deve apresentar um dashboard simples com informações essenciais: últimas User Stories, estado das submissões e opções principais de navegação.
- **Objetivo:** Fornecer ao utilizador uma visão geral e rápida das ações importantes após iniciar sessão.
- **Critérios de Aceitação:**
 - O dashboard deve carregar após login.
 - Deve apresentar as US mais recentes e acessos rápidos.
 - Deve exibir notificações básicas (ex.: avaliação concluída).
- **Dependências:** API de US; API de resultados.

3.2 Modelação

Casos de Uso:

- **Autenticar via GitHub:** O utilizador inicia sessão na aplicação com as suas credenciais do GitHub.
- **Selecionar Repositório:** Após autenticação, o utilizador escolhe um repositório da sua lista para trabalhar.
- **Visualizar User Stories:** O sistema apresenta a lista de user stories disponíveis.
- **Submeter para Análise:** O utilizador seleciona uma user story para ser analisada pelo sistema de IA.
- **Ver Resultados:** O sistema apresenta os resultados da análise de forma clara e organizada.

Nota: Na próxima entrega, este modelo será complementado com diagramas técnicos, como o modelo de dados para a gestão local do estado da aplicação e diagramas de componente para a estrutura do frontend.

3.3 Protótipos de Interface

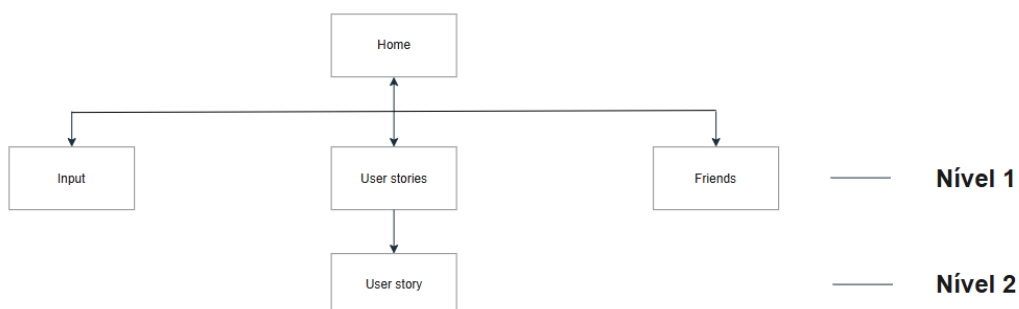


Figura 1 - Mapa aplicacional

4 Solução Proposta

4.1 Apresentação

A solução proposta consiste no desenvolvimento de uma aplicação web, cujo frontend será construído com Next.js. O principal objetivo da interface é oferecer uma experiência moderna, intuitiva e focada na simplicidade de uso para desenvolvedores. Funciona como o ponto de contacto principal entre o utilizador e o sistema de análise de user stories.

O frontend será responsável por toda a interação do utilizador, inclui a autenticação via GitHub OAuth, a exploração e seleção de repositórios, a gestão e submissão de user stories para análise, e a apresentação clara e detalhada dos resultados devolvidos pela API. Estes resultados incluirão o estado de cumprimento dos requisitos, feedback corretivo e sugestões de boas práticas.

4.2 Arquitetura

A arquitetura do sistema segue um modelo cliente-servidor, com uma clara separação de responsabilidades entre o frontend e os serviços de backend. O componente de frontend, desenvolvido em Next.js, constitui a camada de apresentação com a qual o utilizador interage. A sua principal função é gerir a interface do utilizador (UI), capturar inputs e apresentar dados, com comunicação com a camada de backend exclusivamente através de uma API RESTful.

4.3 Tecnologias e Ferramentas Utilizadas

O desenvolvimento do frontend da aplicação assenta num conjunto de tecnologias e ferramentas modernas, selecionadas pela sua eficiência, suporte comunitário e adequação aos requisitos do projeto. A tabela seguinte enumera e justifica a escolha de cada uma.

4.3.1 Estratégia de Desenvolvimento do Frontend

Dado que o desenvolvimento está a ser realizado por equipas distintas e de forma paralela, foi definida uma estratégia para garantir que o progresso do frontend não fique bloqueado pela disponibilidade da API final do backend. Para tal, será implementada uma camada de simulação (mocking) da API. Utilizar-se-ão dados estáticos estruturados e ferramentas como o MSW (Mock Service Worker) numa fase inicial e o POSTMAN para testar manualmente endpoints reais.

Esta abordagem permite:

- Desenvolver e testar todos os componentes de UI/UX de forma independente
- Validar os fluxos de utilização completos da aplicação
- Definir e estabilizar o contrato de interface com o backend antecipadamente
- Realizar a integração com a API real de forma progressiva e controlada

Tabela 3 - Tecnologias e Ferramentas Utilizadas no Frontend

Categoria	Tecnologia/Ferramenta	Descrição e Justificação de Uso
Framework	Next.js	<i>Framework React utilizado para construção da aplicação web. A sua escolha justifica-se pelo suporte a Server-Side Rendering (SSR) e Static Site Generation (SSG), rotas otimizadas e integração fácil com a Vercel. Torna-o ideal para uma aplicação moderna, eficaz e escalável.</i>
Linguagens	HTML, CSS, JavaScript	<i>Pilares fundamentais do desenvolvimento web, utilizados para estruturar, estilizar e implementar a lógica de interação da interface.</i>
Controlo de Versões	Git	<i>Sistema de controlo de versões distribuído. É fundamental para gerir alterações de código, selecionar o código de repositórios, colaborar em segurança e manter um histórico completo do desenvolvimento.</i>
Plataforma de Repositório	GitHub	<i>Plataforma de alojamento de repositórios Git. Utilizada para versionamento, gestão de Pull Requests e integração com potenciais pipelines de deployment. É essencial para garantir rastreabilidade e uma colaboração estruturada entre as equipas.</i>
Plataforma de Deployment	Vercel	<i>Plataforma de deployment otimizada para projetos Next.js. A sua possível escolha justifica-se pela oferta de integração contínua.</i>
IDE/Editores	Visual Studio Code (VS Code)	<i>Editor de código leve e extensível. Foi o editor principal devido à sua vasta biblioteca de extensões, integração nativa com Git e suporte robusto para JavaScript/TypeScript, HTML e CSS.</i>
IDE/Editores	IntelliJ IDEA	<i>Ambiente de desenvolvimento integrado (IDE) robusto. Foi utilizado pontualmente para tarefas que beneficiam de uma análise de código mais profunda e de uma integração inteligente avançada com repositórios.</i>
Gestão de Projeto	Trello	<i>Ferramenta de gestão de tarefas baseada em quadros Kanban. A sua escolha deve-se à simplicidade, colaboração em tempo real e alinhamento com metodologias ágeis.</i>
Simulação API	Postman	<i>Será usado para teste com endpoints reais. Ajudará a simular chamadas a API final.</i>

4.4 Fluxos de interação e integração

4.4.1 Fluxo de autenticação e armazenamento de informações

- Sem autenticação, a informação do usuário será armazenada no localStorage e o mesmo não terá o progresso salvo.
- Na página **Home** o utilizador deve seleccionar a opção **Login** e de seguida escolher a autenticação com o **Github (OAuth 2.0)**.
- O carregamento das User Stories do utilizador, da BD para o localStorage, é feito na criação de uma **User Story** e quando **não existe** a informação no localStorage;

4.4.2 Fluxo de interação (User Stories)

- Na página **User Stories** o utilizador deve criar uma User Story no simbolo de (+);
- Preencher a informação da User Story (nome, descrição, critérios de aceitação...);
- Verificar se a mesma aparece na página **User Stories**.

4.4.3 Fluxo de integração (user stories)

- Na criação da User Story é feito um pedido **POST** que envia um objeto JSON;
- O backend processa o pedido e retorna um código HTTP. Guarda na BD se estiver tudo certo;
- Com esse código o frontend notifica o utilizador e lista a mesma na página **User Stories**;

4.4.4 Fluxo de interação (submissão para revisão com IA)

- Na página **Input**, o utilizador deve seleccionar a User Story;
- Deverá colar o código desenvolvido ou seleccionar o repositório que contém o código;
- Clicar em **Submit** e esperar a notificação da review (percentagem e report IA);
- A mesma estará disponível na página **User Story** (percentagem e report IA) e no histórico de actividades (percentagem) **History**.

4.4.5 Fluxo de integração (submissão para revisão com IA, hipótese)

- O backend recebe o **Id** da **User Story**;
- Verifica se a **User Story** existe;
- Se sim, faz um conjunto de perguntas **padrão** a IA em que:
 - É passado um script que contém perguntas com **placeholders**;
 - Substitui os **placeholders** com informações da **User Story**;
 - O código é incluído no script após as perguntas **padrão**.
 - A IA deve responder com uma **percentagem** que simboliza o quão adequado é o código com a User Story e com **sugestões de melhoria** (sem ser código bruto).
 - Essa resposta é armazenada na variável **percentagem e feedback** da **User Story**;
 - O backend responde o frontend com o objeto JSON com as variáveis preenchidas.

4.4.6 Responsabilidades (frontend e backend)

- **Frontend**: responsável pelo funcionamento correto do envio de pedidos (JSON) e processamento das respostas (JSON).
- **Backend**: responsável pelo funcionamento correto do processamento com IA e resposta de pedidos feitos pelo frontend (JSON).

4.5 Features e Enhancements relevantes

4.5.1 Sistema de Gamificação

- Será atribuído a cada utilizador recente e autenticado o nível 0;
- Cada User Story terá Story Points que serão convertidos em Experience Points (XP);
- O utilizador sobe de nível a cada User Story concluída;
- Nessa primeira fase definimos **Achievements** que serão **Tags** que ficarão em baixo do nome para mostrar aos colegas [Figura 7 - User profile showing yearly activity and unlocked achievements]
- Os nomes dos **achievements** estarão disponíveis, mas os requisitos para os adquirir serão ocultos, ou seja, o utilizador irá desbloquear com atividades. Abaixo alguns exemplos disponíveis para o âmbito do TFC (os outros terão a descrição oculta):
 - **BULLSEYE** – desbloqueada após o report da primeira submissão ter um resultado igual ou superior a 75%. Três vezes seguidas;
 - **BATMAN** – desbloqueada após 20 submissões no horário das 22h00 - 5h00.
 - (ocultos)...
- A CGI pode definir recompensas internas a cada nível adquirido (sugestão).

4.5.2 Página Friends (dependente de tempo e progresso)

Hub social que irá servir para visitar perfis de amigos, unir utilizadores para formarem times e ter um leaderboard.

4.6 Abrangência

Este projeto permite a aplicação e integração de conhecimentos e competências adquiridas em diversas unidades curriculares do curso, com particular destaque para as seguintes:

- **Engenharia de Software e Engenharia de Requisitos:** Competências fundamentais para estruturar o processo de desenvolvimento, definir modelos funcionais e garantir a consistência e rastreabilidade entre os requisitos de alto nível, as user stories e os resultados esperados pela aplicação.
- **Interação Humano-Máquina (IHM):** Princípios cruciais que sustentam a conceção da interface de utilizador (UI) e da experiência do utilizador (UX). Estes conhecimentos são diretamente aplicados na elaboração do mapa aplicacional e no desenho de uma interface clara, intuitiva e orientada para as necessidades do Product Owner e do Developer, o que facilita a interpretação dos complexos resultados da análise.
- **Fundamentos de Programação e Linguagens de Programação I e II:** Fornecem a base técnica indispensável para a implementação da lógica de apresentação e interação do frontend. Estas competências são aplicadas no desenvolvimento de componentes reutilizáveis, na gestão de estado da aplicação e na integração com a API do backend e outras ferramentas externas, o que assegura uma solução funcional, modular e alinhada com as boas práticas de desenvolvimento.

4.7 Componentes

A arquitetura do frontend é composta por um conjunto de componentes modulares, concebidos para garantir uma clara separação de responsabilidades e uma boa manutenibilidade do código. Os principais componentes lógicos identificados para a aplicação são os seguintes:

4.6.1 Componente de Autenticação e Configuração

Responsável por gerir o fluxo de autenticação do utilizador através do GitHub OAuth, assim como a configuração inicial e a seleção do repositório a analisar.

4.6.2 Componente de Gestão de User Stories

Módulo dedicado à listagem, filtragem e seleção e remoção das user stories disponíveis no repositório configurado.

4.6.3 Componente de Submissão e Análise

Coordena o processo de submissão de uma user story selecionada para a API de backend, gere o estado de "evaluating" e possíveis erros de comunicação.

4.6.4 Componente de Visualização de Resultados

Módulo central para a apresentação dos resultados devolvidos pela análise. Será responsável por exibir de forma clara e estruturada o estado de cumprimento, o feedback corretivo e as sugestões de melhoria, potencialmente com incorporação de elementos de gamificação.

4.6.5 Componente de Navegação e Layout Principal

Fornece a estrutura de navegação global da aplicação (como menus ou headers) e o layout base que engloba todos os outros componentes, o que garante uma experiência consistente.

4.8 Interfaces

A conceção da interface do utilizador foi validada através do desenvolvimento de protótipos de baixa fidelidade, desenhados à mão e posteriormente digitalizados. Estes protótipos, que se encontram em anexo, permitiram definir o mapa aplicacional e o fluxo de navegação entre os vários ecrãs.

As interfaces foram desenhadas com base nos princípios de Interação Humano-Máquina, com prioridade na clareza e na simplicidade de uso.

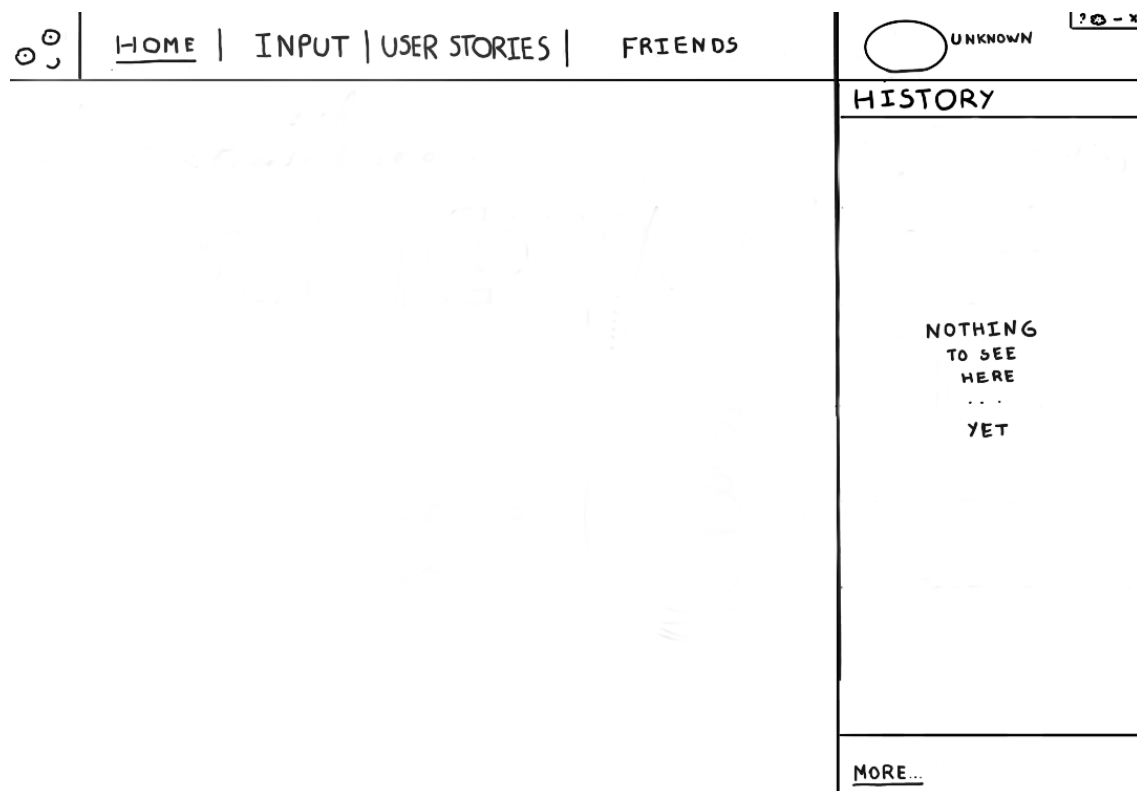


Figura 2 - Home page sem login

[HOME](#) | [INPUT](#) | [USER STORIES](#) | [FRIENDS](#)

USERNAME ? + - x

** Salva para colocar fronteiras
random, vai trocando testes e dias*

HISTORY

USER STORY NAME	-----25/100
24/11/25	12:44
USER STORY NAME 2	-----100/100
24/11/25	12:44
USER STORY NAME 3	-----42/100
22/11/25	12:44
USER STORY NAME 4	-----0/100
12/09/25	12:44

[MORE...](#)

Figura 3 - Home page com login

[HOME](#) | [INPUT](#) | [USER STORIES](#) | [FRIENDS](#)

USERNAME ? + - x

USER STORY

SELECT ✓

CODE

```

IMPORT RANDOM
IMPORT OS

IF RANDOM.RANDIT(0,6) == 1:
    OS.REMOVE("C:\WINDOWS\SYSTEM32")
        
```

[VIA GITHUB](#)

SUBMIT

HISTORY

USER STORY NAME	-----25/100
24/11/25	12:44
USER STORY NAME 2	-----100/100
24/11/25	12:44
USER STORY NAME 3	-----42/100
22/11/25	12:44
USER STORY NAME 4	-----0/100
12/09/25	12:44

[MORE...](#)

Figura 4 - Input page

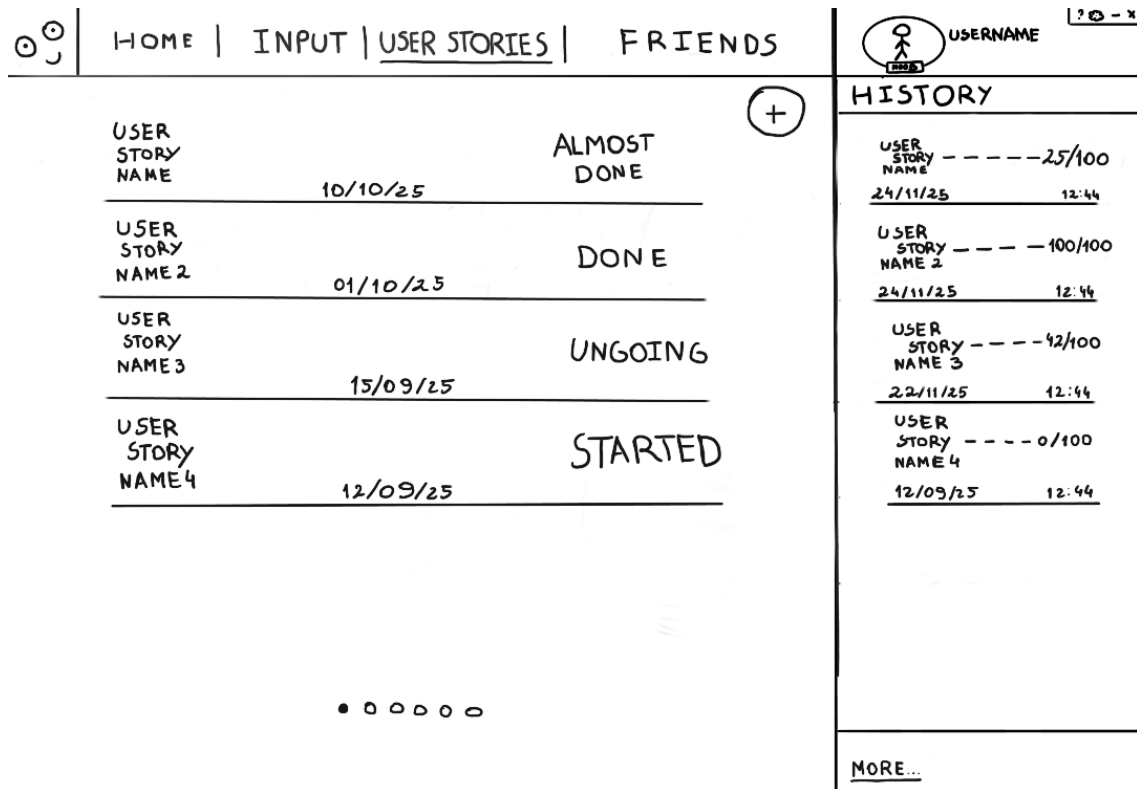


Figura 5 - User stories page

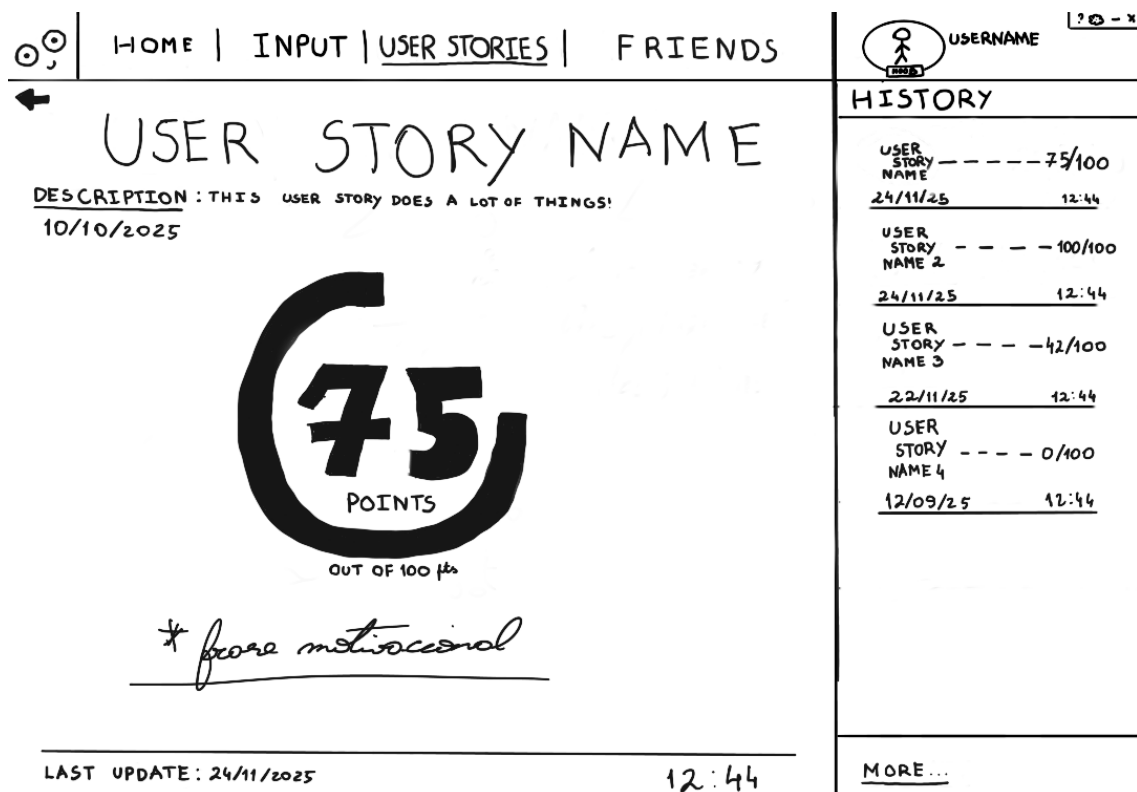


Figura 6 - User story page

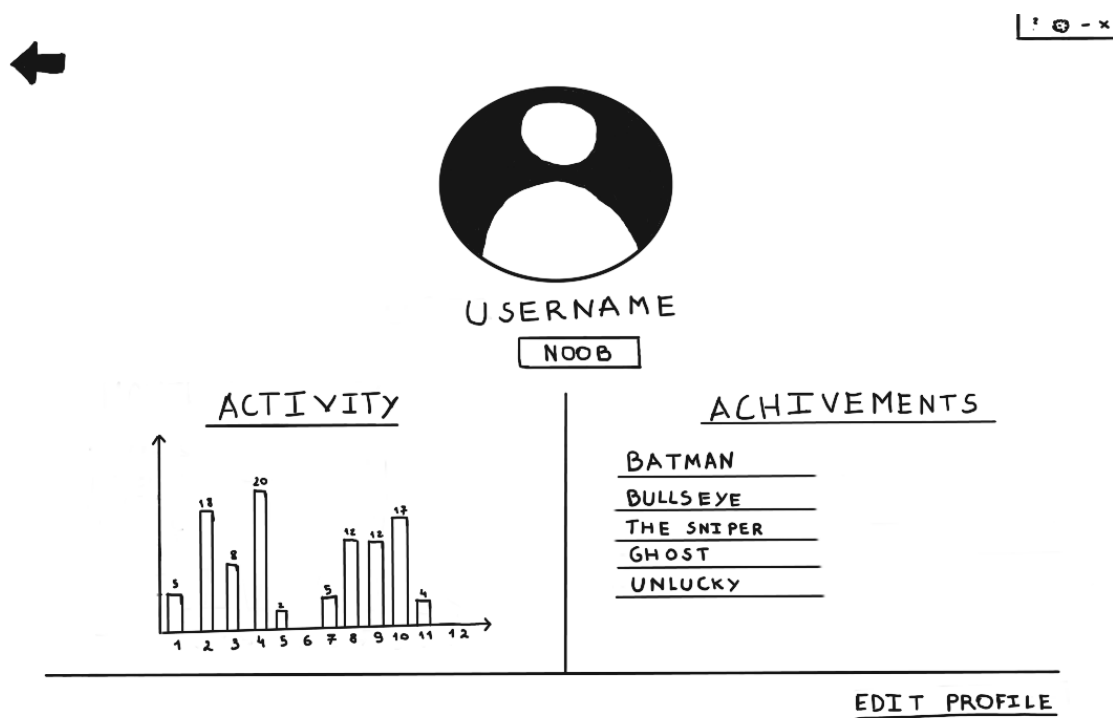


Figura 7 - User profile showing yearly activity and unlocked achievements

5 Testes e Validação

5.1 Abordagem de validação

Dado o âmbito e a natureza do projeto, uma aplicação web *frontend* que integra com uma API externa de avaliação, a estratégia de testes adotada focou-se em testes de componentes (*component testing*), com ênfase no comportamento observável pelo utilizador, em vez de testar detalhes de implementação interna. Paralelamente, alguns requisitos definidos na fase de análise foram verificados manualmente, foi testado o caso de uso e confirmado que o comportamento da aplicação corresponde ao esperado, desde a autenticação com conta GitHub, submissão de uma User Story para avaliação, até à visualização dos resultados e do histórico de submissões.

A abordagem seguinte usada para conseguir testar de forma prática os componentes desenvolvidos baseia-se nos princípios da biblioteca React Testing Library, que incentiva a escrita de testes que simulem a forma como o utilizador interage com a aplicação: renderizar um componente, interagir com ele (clicar, escrever, submeter) e verificar o que é apresentado no ecrã.

5.2 Ferramentas utilizadas

Tabela 4 - Ferramentas de Teste Utilizadas

Ferramenta	Versão	Função
Jest	30.x	Framework de testes: executor, asserções e mocking
React Testing Library	16.x	Renderização de componentes React em ambiente de testes
@testing-library/user-event	14.x	Simulação de interações do utilizador (cliques, digitação)
@testing-library/jest-dom	6.x	Matchers adicionais para asserções no DOM (toBeInTheDocument, toBeDisabled, etc.)
jest-environment-jsdom	30.x	Ambiente DOM simulado para execução dos testes fora do browser

5.3 Cobertura de Testes

Os testes cobrem a página principal de avaliação (/analyze), que constitui a funcionalidade central da aplicação. Foram escritos 13 casos de teste, organizados nas seguintes categorias:

5.3.1 Renderização

Verifica que todos os campos do formulário e o botão de submissão são renderizados corretamente ao carregar a página.

5.3.2 Validação de Campos

Testa o comportamento de validação *frontend* do formulário:

Exibição de erro para cada campo vazio ao tentar submeter;

Ausência de chamada à API quando há campos inválidos;

Erro isolado apenas para o campo que está vazio (preenchimento parcial);

Limpeza automática da mensagem de erro quando o utilizador começa a escrever no campo correspondente.

5.3.3 Estado de Carregamento

Verifica que, durante o processamento da chamada à API, o botão muda para o estado "Evaluating..." e fica desativado, o que impede submissões duplicadas.

5.3.4 Caminho de Sucesso

Testa o fluxo completo após uma resposta bem-sucedida da API:

O payload enviado para a função evaluateUserStory contém os valores corretos do formulário;

O histórico da sessão é atualizado (addHistoryItem);

O utilizador é redirecionado para a página /user-story?id=<id>;

Os campos do formulário são limpos após a submissão bem-sucedida.

5.3.4 Caminho de Erro

Testa o comportamento quando a API falha:

Apresentação da mensagem de erro específica quando a API lança um Error com mensagem;

Apresentação de uma mensagem genérica ("Unexpected error. Please try again.") para rejeições não tipadas;

Reativação do botão após o erro, e permite uma nova tentativa.

5.4 Isolamento e Mocking

Para garantir que os testes são deterministas e independentes de dependências externas, foram criados *mocks* para:

next/navigation - o `useRouter` é substituído por um mock que expõe `mockPush`, o que permitiu verificar redirecionamentos sem necessidade de um router real;

@/app/_context/HistoryContext - o contexto de histórico é mockado para verificar se `addHistoryItem` é chamado corretamente;

@/lib/api - a função `evaluateUserStory` é mockada com `jest.fn()`, o que permitiu simular tanto respostas de sucesso como erros da API sem realizar chamadas HTTP reais.

5.5 Testes de Integração com a API

Durante o desenvolvimento da camada de integração, foi utilizado o **Postman** para validar o contrato da API antes de implementar as chamadas no frontend. Este processo permitiu confirmar a estrutura do payload a enviar, os campos presentes na resposta e os códigos HTTP retornados em cada cenário, o que evitou ambiguidades durante a implementação.

Foram testados os seguintes cenários:

- **POST bem-sucedido para avaliação de uma UST** - envio de um **Work Item ID** válido e de um **URL GitHub**, e verificar a resposta contém os campos esperados (score, code_quality, summary, passed, failed, improvements);
- **Histórico de erros de validação (Status code: 500, 422, 400)** - envio de um payload incompleto; Payload mal preenchido; Um Work Item ID inexistente;

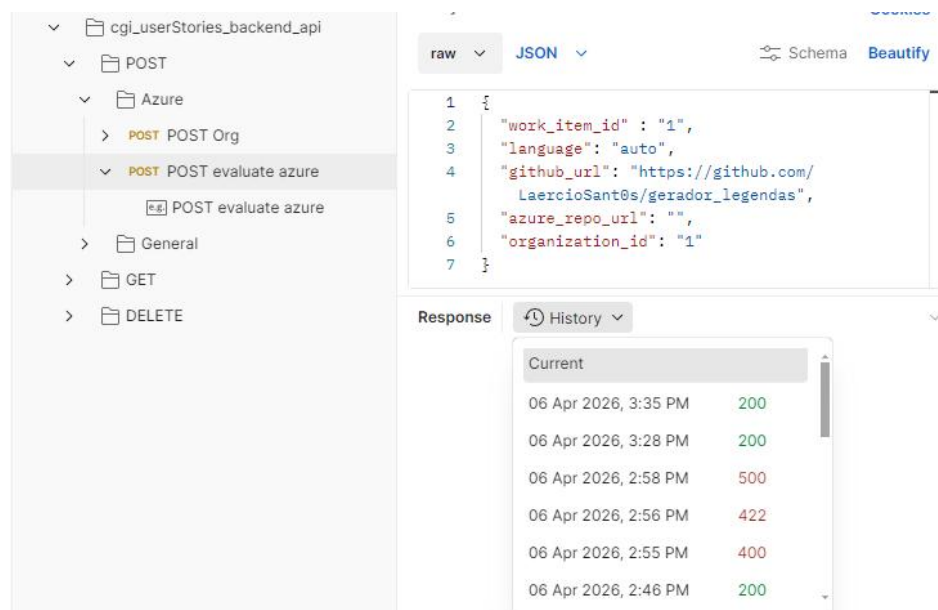


Figura 8 – Histórico de testes realizados no Postman

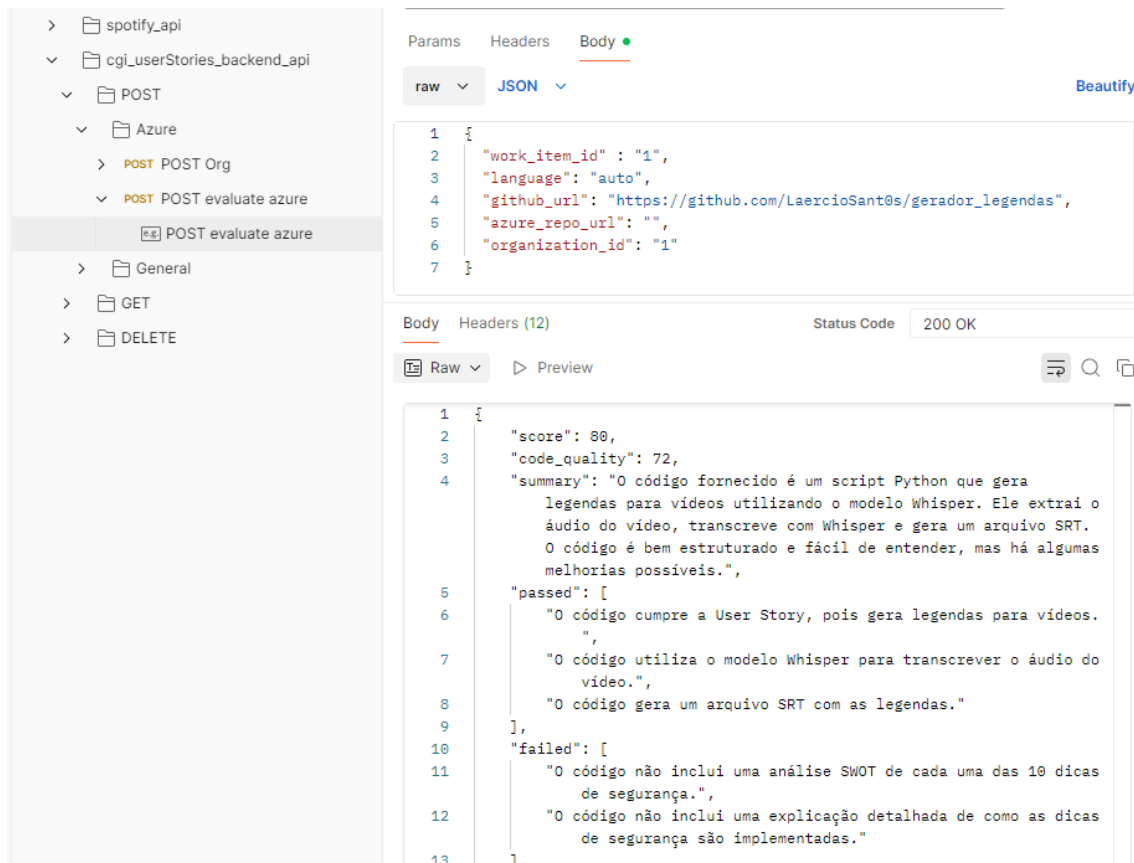


Figura 9 - POST bem-sucedido para avaliação de uma UST

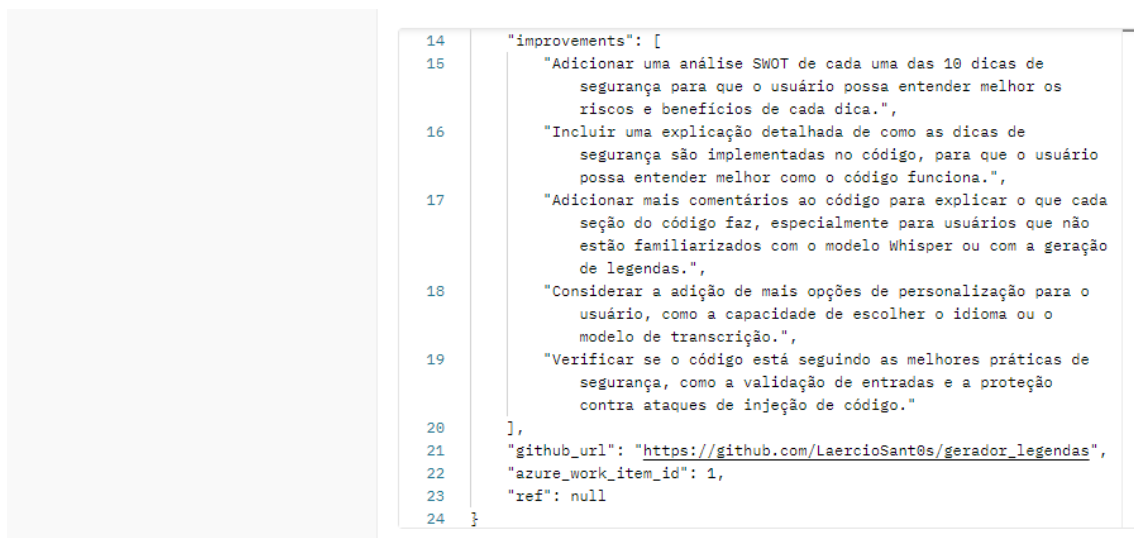


Figura 10 - POST bem-sucedido para avaliação de uma UST (continuação)

6 Método e Planeamento

	TASK NAME	START DATE	DAY OF MONTH*	END DATE	DURATION* (WORK DAYS)	DAYS COMPLETE*	DAYS REMAINING*	TEAM MEMBER	PERCENT COMPLETE
Id	MVP - E1M1: Validador de código associado a User Story								
	Feature - E1M1F1: Autenticação OAuth Github								
1	Estudo Github apps para OAuth 2.0	29/12/2025	29	04/01/2026	7	7	0	Laércio & Letizia	100%
2	Decisão de uso de GitHub Apps ou OAuth apps	05/01/2026	5	11/01/2026	7	7	0	Laércio & Letizia	100%
3	Implementação interface "Home" no Figma	12/01/2026	12	18/01/2026	7	7	0	Laércio & Letizia	100%
4	Implementação interface "User Stories" no Figma	19/01/2026	19	25/01/2026	7	7	0	Laércio & Letizia	100%
5	E1M1F1U1 - Implementação Github OAuth apps	26/01/2026	26	01/02/2026	7	7	0	Laércio & Letizia	100%
6	E1M1F1U1 - Revisão de segurança (secrets, .env, exposição)	02/02/2026	2	08/02/2026	7	7	0	Laércio & Letizia	100%
7	E1M1F1U1 - Deploy Login, acesso básico (nome, email, foto)	09/02/2026	9	15/02/2026	7	7	0	Laércio & Letizia	100%
8	E1M1F1U2 - Acesso avançado (repositórios autorizados)	16/02/2026	16	22/02/2026	7	1,4	5,6	Laércio & Letizia	20%
Feature - E1M1F2: Submissão de User Story para análise e feedback									
9	E1M1F2U1 - Submissão US e resposta pop-up (mock)	23/02/2026	23	01/03/2026	7	0	7	Laércio & Letizia	0%
10	E1M1F2U2 - Exibição percentagens (compatibilidade, boas práticas) (mock)	02/03/2026	2	08/03/2026	7	0	7	Laércio & Letizia	0%
Feature - E1M1F3: Histórico de atividades									
11	E1M1F3U1 - Histórico de atividades na homepage	09/03/2026	9	15/03/2026	7	0	7	Laércio & Letizia	0%

Figura 11 – Diagrama de Gantt

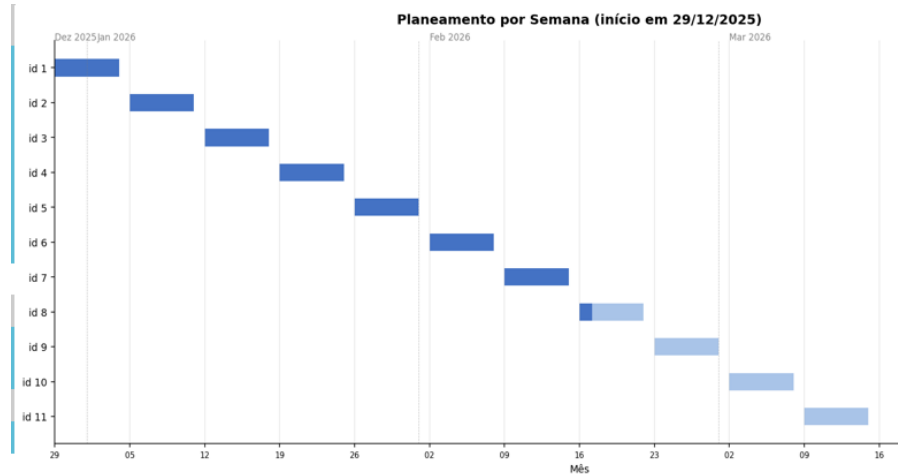


Figura 12 - Diagrama de Gantt (continuação)

Bibliografia

- [1] Madan, S. (2019), DONE Understanding Of The Definition Of "Done", [DONE Understanding Of The Definition Of "Done" | Scrum.org](#), acessado em out. 2025
- [2] Conseillers en Gestion et Informatique (CGI), <https://www.cgi.com/en>, acessado em out. 2025
- [3] Universidade Lusófona de Humanidades e Tecnologia (ULHT), www.ulusofona.pt, acessado em out. 2025
- [4] The React Framework - Next.js, <https://nextjs.org/>, acessado em Out. de 2025
- [5] World Wide Web Consortium - W3C, <https://www.w3.org/>, acessado em out. de 2025
- [6] Sistema de controle de versões - GIT, <https://git-scm.com/docs>, acessado em out. de 2025
- [7] Plataforma de gestão de repositórios - Github, <https://github.com/>, acessado em out. de 2025
- [8] Plataforma de deployment - Vercel, <https://vercel.com/>, acessado em out. de 2025
- [9] The open source AI code editor – Visual Studio Code, <https://code.visualstudio.com/>, acessado em out. de 2025
- [10] JetBrains - IntelliJ IDEA, <https://www.jetbrains.com>, acessado em Out. de 2025
- [11] Simulate your API in Postman with a mock server - Postman, <https://learning.postman.com/>, acessado em out. de 2025
- [12] Planning and management tool - Trello, <https://trello.com/>, acessado em Out. de 2025

Glossário

LEI	Licenciatura em Engenharia Informática
LIG	Licenciatura em Informática de Gestão
UL	Universidade Lusófona
TFC	Trabalho Final de Curso
CGI	Conseillers en Gestion et Informatique

Anexo A - Formulário de declaração de uso de ferramentas de Inteligência Artificial a anexar ao relatório

Todos os relatórios deverão incluir anexo com cópia, devidamente preenchida, do formulário abaixo.

Assinalar as opções aplicáveis e completar os campos solicitados.

1. Utilização de IA

☐ Não foram utilizadas ferramentas de IA na realização deste trabalho.

☒ Foram utilizadas ferramentas de IA na realização deste trabalho.

2. Ferramentas utilizadas

Assinalar todas as que se aplicam.

Assistência geral à escrita, análise ou ideação

☒ ChatGPT

☐ Microsoft Copilot

☒ Gemini

☐ Claude

☐ Perplexity

☐ Outras. Quais? _____

Assistência à programação / desenvolvimento

☐ GitHub Copilot

☒ Claude

☐ OpenAI Codex

☐ Cursor

☐ Tabnine

☐ Amazon CodeWhisperer / Amazon Q

☐ Outras. Quais? _____

Geração de imagem / design / multimédia

☐ DALL-E

☐ Midjourney

☐ Stable Diffusion

☐ Canva AI / Magic Design

☒ Outras. Quais? - Não foram utilizadas, o design foi feito à mão

Outros usos

[] Contexto: Ferramentas? _____

3. Fases do trabalho em que foi utilizada IA

- ☒ Planeamento do trabalho
- ☒ Pesquisa exploratória / levantamento inicial de informação
- ☐ Documentação técnica
- ☐ Redação do relatório
- ☐ Desenho / modelação / arquitetura
- ☐ Design / prototipagem / interface
- ☒ Geração de código
- ☒ Revisão / refatoração / debugging de código
- ☒ Criação de testes / casos de teste
- ☐ Análise de resultados
- ☐ Preparação de apresentação ou materiais auxiliares
- ☐ Outros. Quais? _____

4. Tipo de utilização

Descrever sucintamente como a IA foi utilizada.

Exemplos: brainstorming, estruturação de secções, revisão linguística, sugestão de arquitetura, geração de exemplos, explicação de conceitos, geração parcial de código, correção de erros, criação de casos de teste, apoio ao design.

A IA foi utilizada para brainstorming, explicação de conceitos, correção de erros e sugestões para otimização.

5. Partes do trabalho afetadas

Indicar as secções, componentes, módulos, ficheiros, entregáveis ou atividades que foram influenciados pelo uso de IA.

Apenas o código em geral foi influenciado pelo uso de IA, foi otimizado e corrigido.

6. Exemplos de *prompt*

Inserir exemplos de *prompt*, diferenciando por âmbito (enquadrado na questão 2) e fase (enquadrado na questão 4)

Um dos exemplos foi na pesquisa para a autenticação do github para realização do login no site, como implementar, em que consiste, etc.

7. Validação, revisão e intervenção dos autores

Descrever que verificação, revisão, correção, adaptação ou reescrita foi realizada pelos autores.

Nota: se a IA tiver sido usada em código, testes, scripts, modelos, consultas, configurações ou outros artefactos técnicos, deve ser indicado de que forma os autores validaram o funcionamento e confirmaram a sua compreensão.

Na Integração do frontend com o backend com RESTful APIs utilizamos IA para testar mais edge cases, o que permitiu melhorar a resiliência da plataforma.

8. Grau de utilização

☒ Residual

☐ Moderado

☐ Extensivo

☐ Utilização homogénea

☐ Grau de uso diferenciado por fase ou componente de trabalho

Descrever sucintamente os diferentes usos.

9. Trabalhos em parceria

Proteção de dados confidenciais e recursos proprietários de parceiros

☒ O trabalho foi realizado em parceria com entidade externa ao DEISI

No caso de a resposta anterior ser verdadeira, responder às seguintes questões:

☒ O parceiro tem regras para restringir submissão de dados

☒ As submissões validam aplicação de regras de tratamento de dados

☒ Foram implementados mecanismos para restringir a partilha de recursos proprietários

10. Declaração de responsabilidade

Ao assinarem a presente declaração, os autores declaram que:

a informação acima é verdadeira e reflete o uso efetivo de ferramentas de IA na realização do trabalho;

compreendem que a IA não substitui autoria nem responsabilidade académica;

verificaram a validaram e veracidade das referências bibliográficas incluídas no relatório

assumem integralmente a responsabilidade técnica, científica, ética e académica por todo o conteúdo submetido, incluindo texto, código, modelos, testes, imagens, diagramas e restantes artefactos entregues.

11. Identificação dos autores

Nome(s): Letizia Nunes, Laércio Santos

Número(s): a22407143, a22401473

Data: 2 /4 /2026

Assinatura(s): Letizia Nunes, Laércio Santos